



RED DE EMERGENCIA VALLE

REV — Municipalidad de Valle del Sol

CONECTIVIDAD QUE SALVA VIDAS

Innovación que resguarda el mañana · DSY1106 EVA2 · Mayo 2026

NICOLÁS BARRA

GIANNINA GUERRERO

PROF. ISRAEL ALEJANDRO VILLAGRA RIQUELME

Plataforma de **misión crítica** en microservicios Spring Cloud. Esta presentación resume el **informe técnico integral** con evidencias en código, capturas del dashboard y arquitectura desplegada en Docker.

Entregables EVA2 — Segunda evaluación

DOCUMENTACIÓN E IMPLEMENTACIÓN ALINEADA AL INFORME INTEGRAL

INFORME INTEGRAL

14 capítulos · fig. 1–20

[informe-tecnico-integral-rev.html](#)

ECOSISTEMA

BFF + 3 MS + Gateway

Keycloak · Eureka · Docker

FRONTEND

Dashboard operacional

[frontend/rev-dashboard/](#)

ARQUETIPO

Maven custom

[rev-microservice-archetype/](#)

EVIDENCIAS

Capturas UX + código

[docs/informe-evidencias/](#)

GIT + CI

main / dev

commits [TIPO]: · GitHub Actions

Metodología: cada afirmación del informe se vincula a archivos del monorepo. Las figuras 14–16b son **screenshots reales** con datos del stack local.

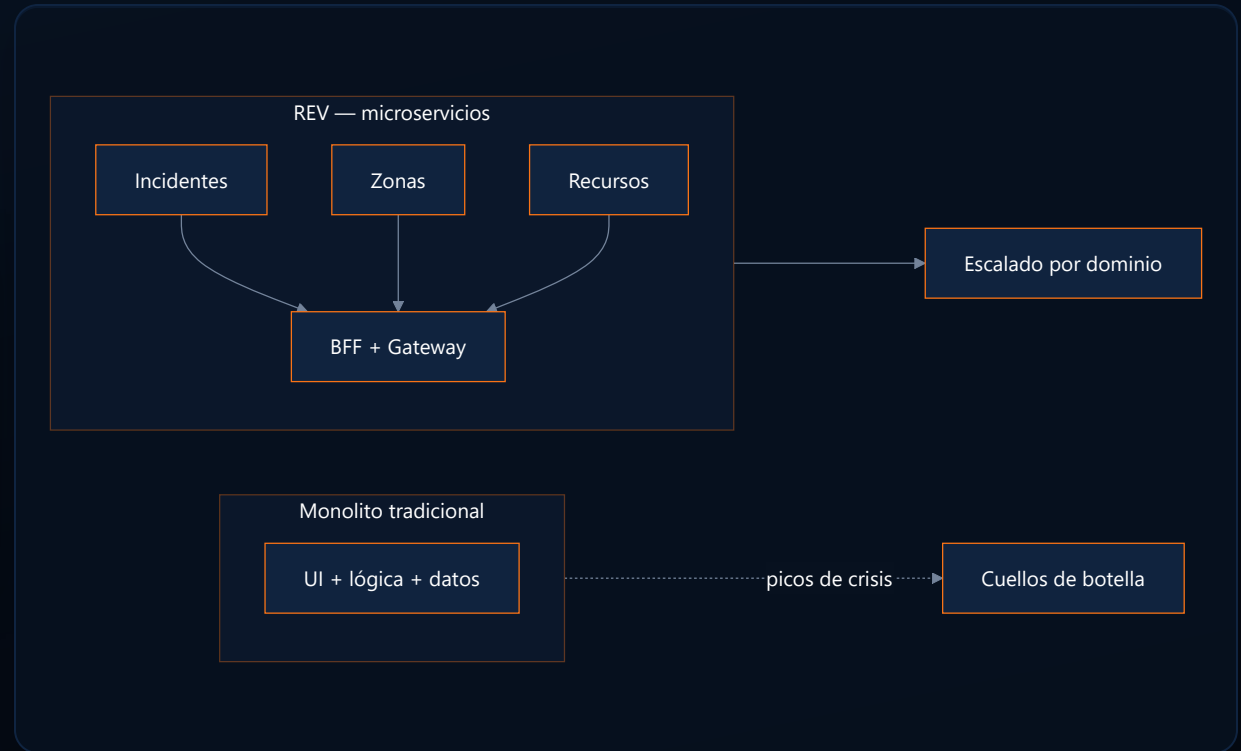
Problema identificado

¿POR QUÉ ES NECESARIO REV?

Los municipios enfrentan **picos impredecibles de demanda** durante incendios, incidentes urbanos y emergencias estructurales.

LIMITACIÓN	IMPACTO
Acoplamiento monolítico	Un fallo tumba todo
Escalado uniforme	No prioriza incidentes
Interfaces fragmentadas	Despachador pierde tiempo
Canales ciudadanos lentos	Demora activación brigadas

El problema no es «falta de software», sino **arquitectura no adaptable** a la urgencia municipal.



Objetivos del proyecto

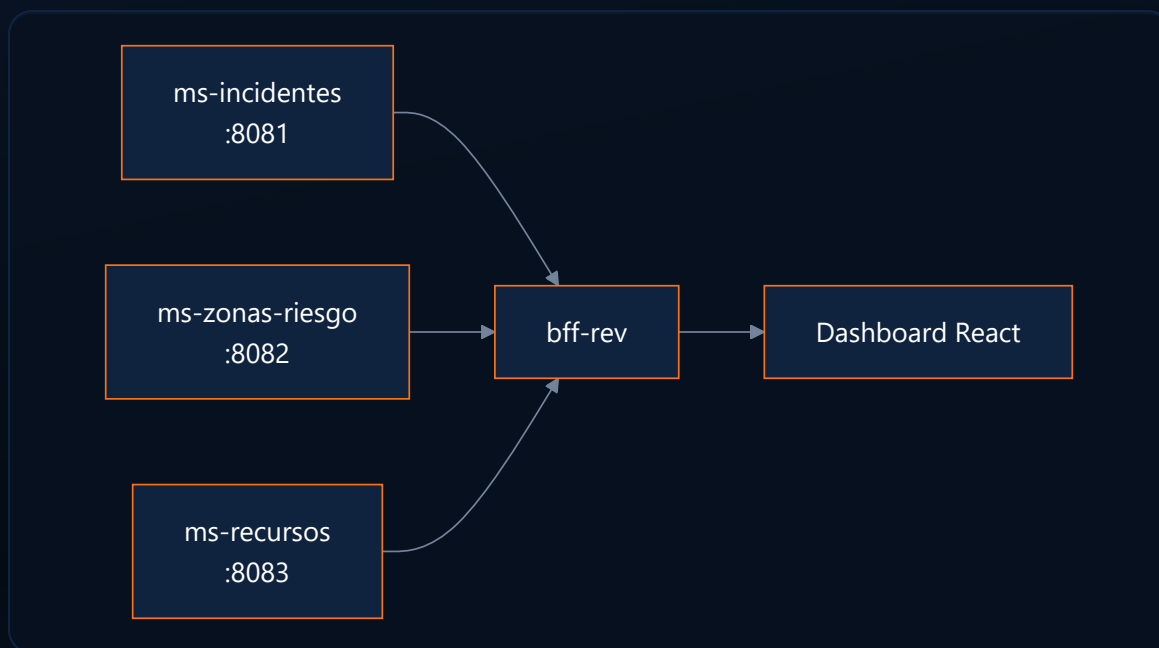
ALINEACIÓN OBJETIVO ↔ ARQUITECTURA

TIPO	OBJETIVO	COMPONENTE QUE LO MATERIALIZA
General	Plataforma integral de gestión de emergencias municipales	Monorepo: React + Gateway + BFF + 3 MS
Específico 1	Gestionar ciclo de vida de incidentes con reglas de negocio	<code>ms-incidentes</code> + Factory/State
Específico 2	Evaluar riesgo territorial por coordenadas	<code>ms-zonas-riesgo</code> + PostGIS
Específico 3	Coordinar brigadas, vehículos y herramientas	<code>ms-recursos</code> + asignación vía BFF
Específico 4	Vista unificada para el despachador	<code>bff-rev</code> + <code>DashboardFacadeService</code>
Específico 5	Canal ciudadano sin autenticación	Portal <code>/portal</code> + <code>POST /api/public/incidentes</code>

Cada objetivo específico corresponde a un **bounded context** con base de datos propia. El BFF agrega; los MS deciden.

Visión general de REV

PROPÓSITO, ACTORES Y DOMINIOS

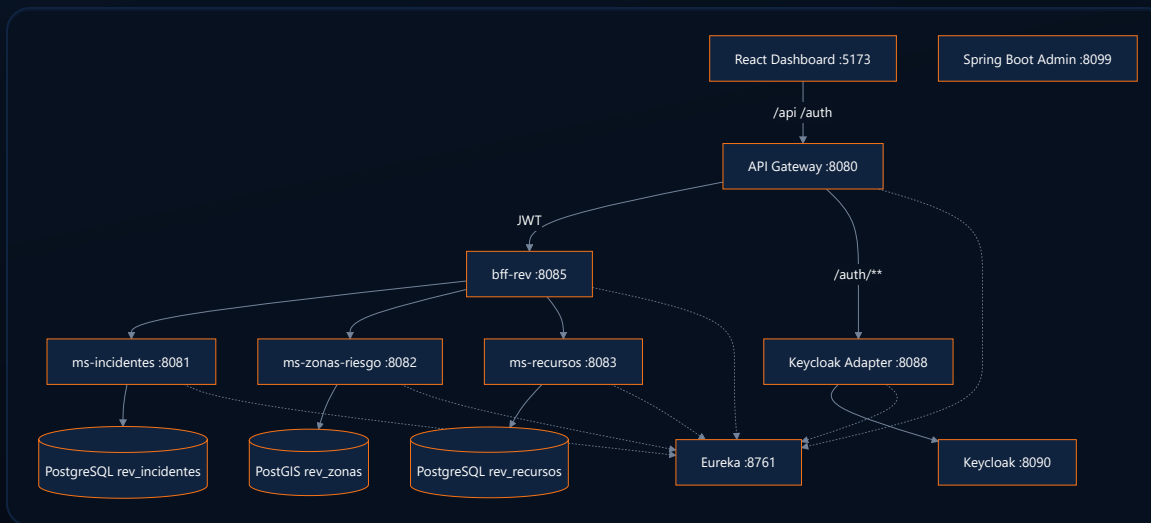


ACTOR	ROL
Despachador	Crea incidentes, asigna recursos
Brigadista	Consulta estado y riesgo
Administrador	Operación + Keycloak
Ciudadano	Reporta vía portal público

Tres dominios **sin BD compartida**. El BFF entrega **DashboardResponse** unificado.

Arquitectura general

ECOSISTEMA VERIFICADO EN EL MONOREPO



PERÍMETRO
Gateway + JWT

DATOS
3 BD aisladas

DISCOVERY
Eureka lb://

MONITOR
SBA :8099

Microservicios implementados

RESPONSABILIDAD, DATOS Y BENEFICIO DE LA SEPARACIÓN

MICROSERVICIO	PUERTO	BD	RESPONSABILIDAD
ms-incidentes	8081	rev_incidentes	Ciclo de vida del incidente
ms-zonas-riesgo	8082	rev_zonas	Territorio y evaluación de riesgo
ms-recursos	8083	rev_recursos	Logística operacional

MS-INCIDENTES

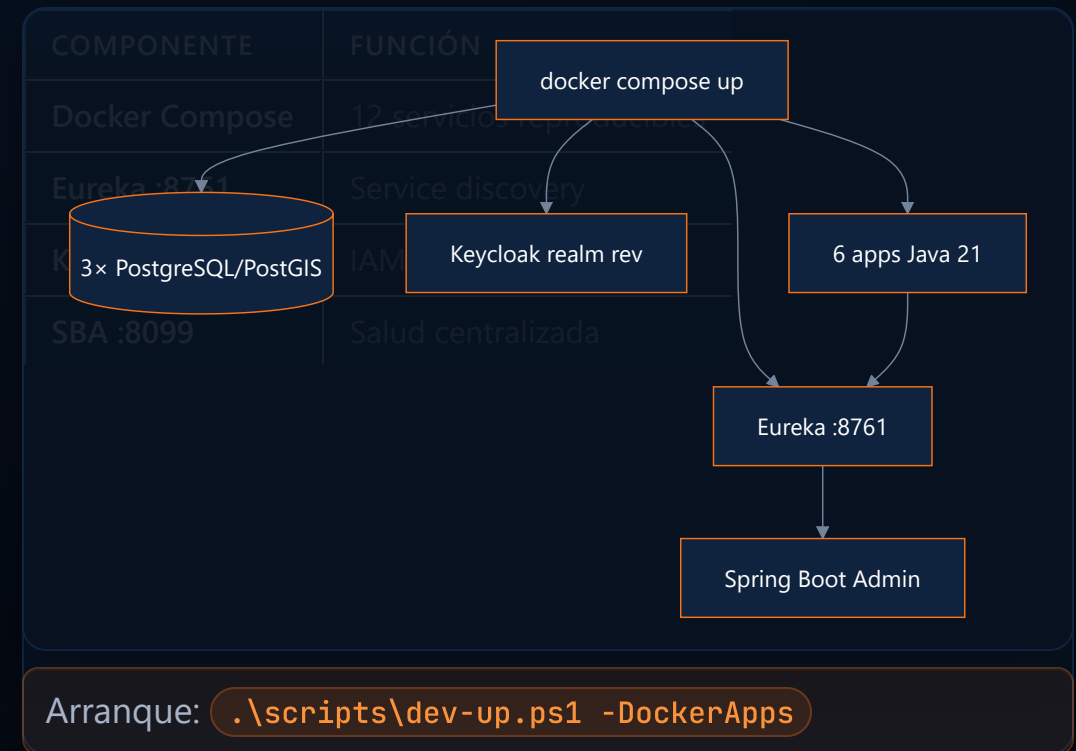
Reglas de transición encapsuladas (Factory + State). Cambios en estados no afectan zonas ni recursos.

MS-ZONAS-RIESGO

PostGIS + adaptador climático (WeatherDataPort). Evolución territorial sin tocar incidentes.

ms-recursos: asignaciones con incidente_id UUID — desacoplamiento cross-service sin FK entre bases de datos.

COMPONENTES TRANSVERSALES DEL ECOSISTEMA



Patrones arquitectónicos

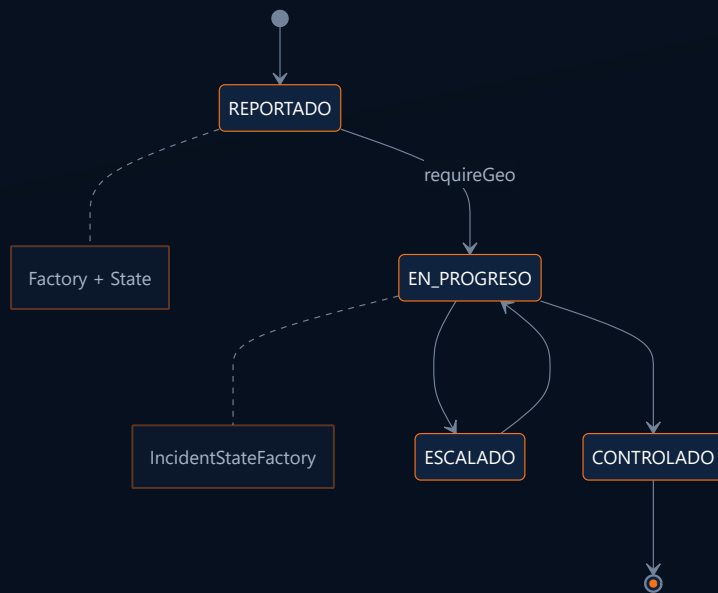
DE LA TEORÍA A LA IMPLEMENTACIÓN REV

PATRÓN	APLICACIÓN EN REV	BENEFICIO
Microservices	3 MS + BFF + Gateway	Escalado independiente
API Gateway	<code>api-gateway</code> :8080	Seguridad centralizada
BFF	<code>DashboardFacadeService</code>	Una llamada al dashboard
Service Discovery	Eureka + <code>lb://BFF-REV</code>	Sin hardcodear hosts
Circuit Breaker	Resilience4j en BFF	Operación parcial ante fallos
Database per Service	3 PostgreSQL/PostGIS	Autonomía de datos

Cache-aside: `ZonaRiesgoCache` sirve datos de riesgo cuando `ms-zonas-riesgo` no responde.

Patrones de diseño implementados

TRAZABILIDAD CLASE → PROBLEMA → BENEFICIO

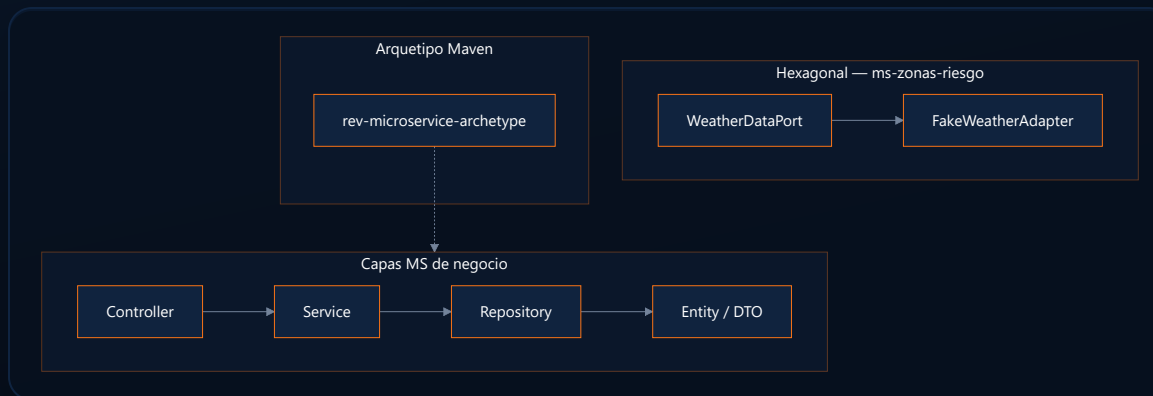


PATRÓN	IMPLEMENTACIÓN
Factory + State	<code>IncidentStateFactory</code>
Adapter	<code>FakeWeatherAdapter</code>
Facade	<code>DashboardFacadeService</code>
Repository	Spring Data JPA

ReportadoState exige georreferenciación para pasar a **EN_PROGRESO**.

Arquetipos utilizados

ESTRUCTURA REUTILIZABLE DEL MONOREPO

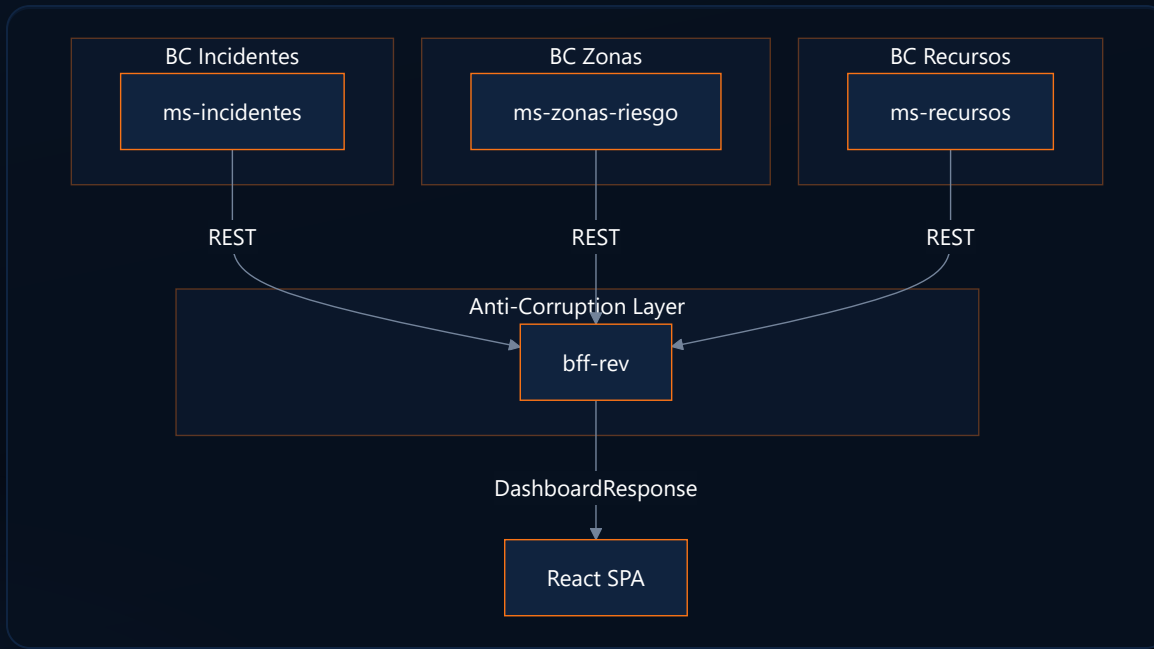


CAPA	EJEMPLO
Controller	<code>IncidenteController</code>
Service	<code>IncidenteService</code>
Repository	<code>IncidenteRepository</code>

Estandariza nuevos MS municipales sin reconfigurar Eureka, Actuator ni Flyway.

DDD y Bounded Contexts

TRES SUBDOMINIOS = TRES MICROSERVICIOS AUTÓNOMOS



VENTAJA	EJEMPLO REV
Lenguaje ubicuo	«Estado» vs «Nivel»
Evolución independiente	CRUD zonas sin migrar incidentes
Fallas contenidas	Circuit Breaker por MS

Contrato UI:

```
{ incidente, zonaRiesgo, recursos, degraded }
```

Frontend y experiencia de usuario

MÓDULOS OPERATIVOS VERIFICADOS EN UI

MÓDULO	RUTA	CAPACIDAD
Inicio	<code>/inicio</code>	KPIs y panorama
Despacho	<code>/</code>	Tabla activos y alertas
Incidentes	<code>/incidentes</code>	Filtros, cards, rail
Zonas	<code>/zonas</code>	Mapa Leaflet
Recursos	<code>/recursos</code>	Brigadas y vehículos
Portal	<code>/portal</code>	Reporte ciudadano

- Una llamada al BFF — `fetchDashboard()`
- **ModuleHub** — KPIs + toolbar + rail
- **StateView** — loading / error / empty
- **Lenguaje operacional** — «Con avisos», «Información parcial»

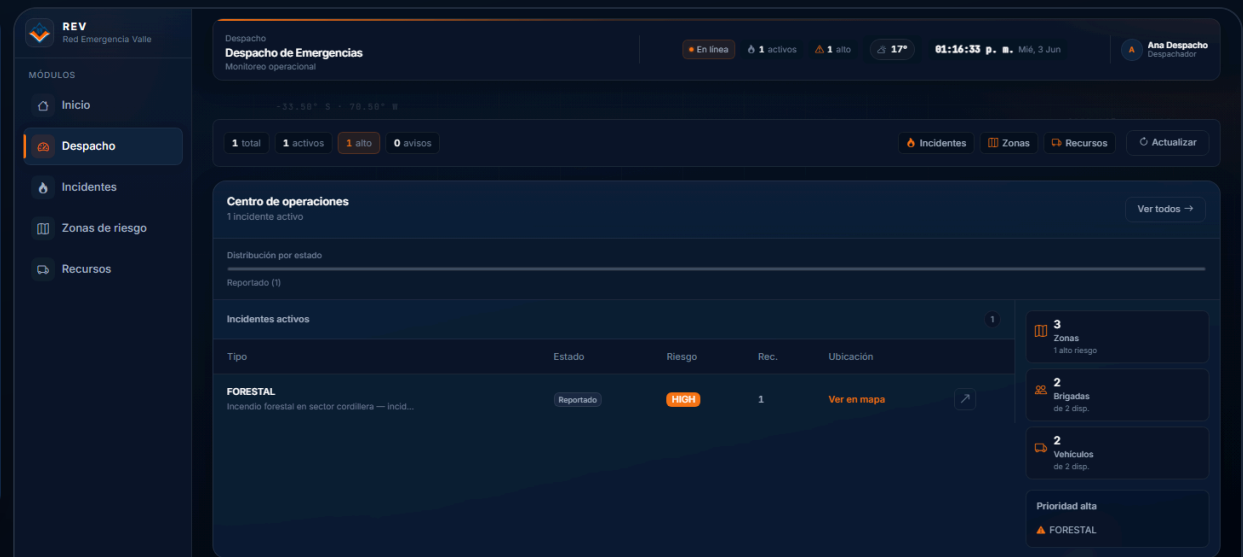


Fig. 14 — Panel Despacho (/)

Reporte público y canal ciudadano

INTEGRACIÓN FRONTEND ↔ BFF ↔ MS-INCIDENTES

CANAL	ruta	BACKEND
Login — Reportar	/login	POST público
Portal	/portal	Sin registro
Despacho	/api/**	JWT

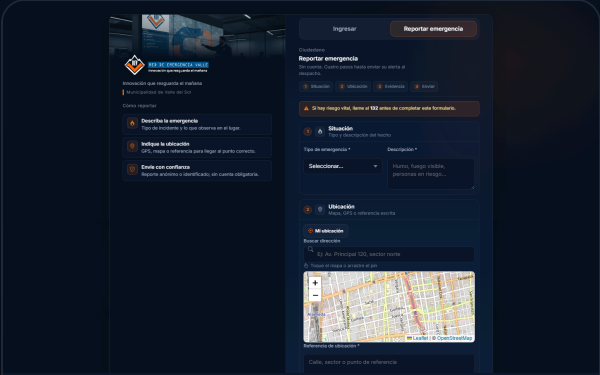
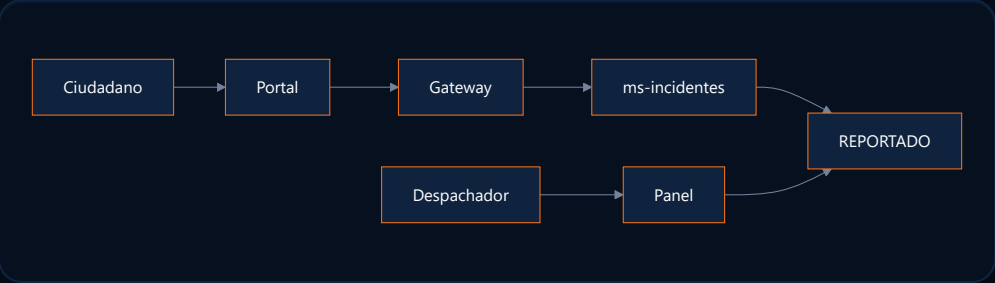


Fig. 16 — Reporte georreferenciado

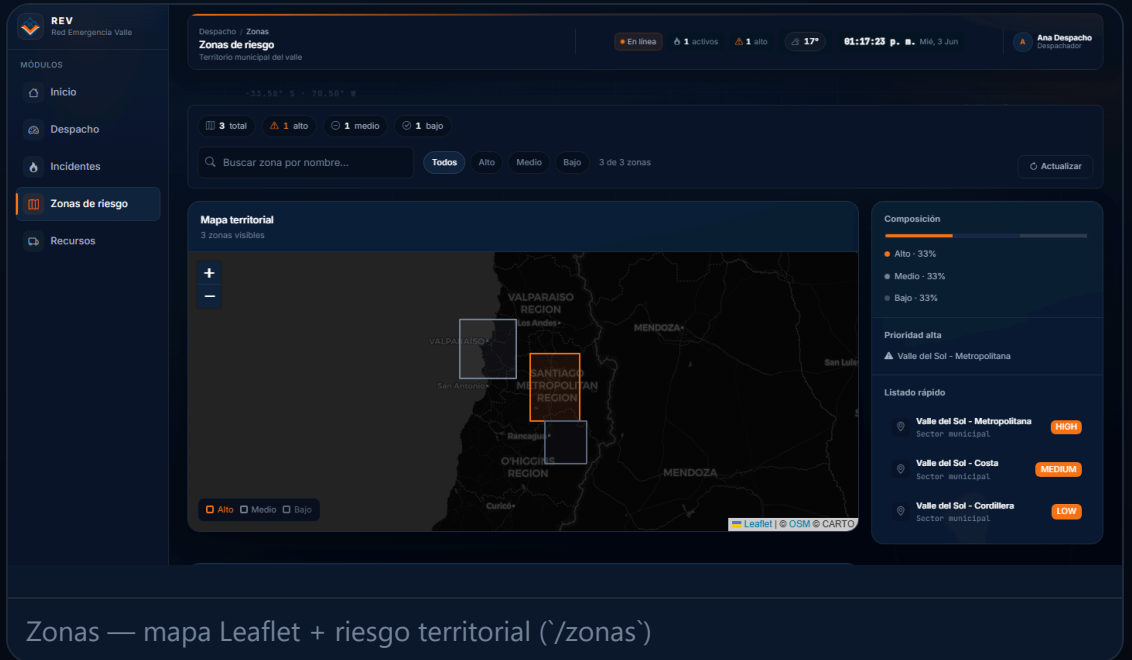
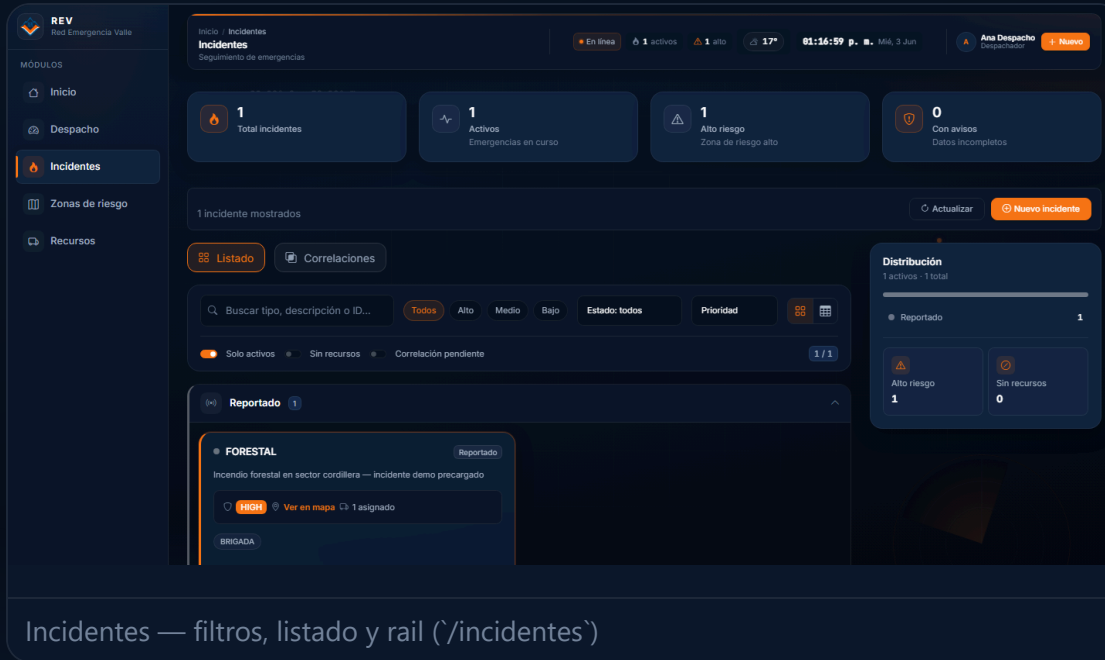


Fig. 16b — Portal ciudadano

El reporte público crea incidentes en **REPORTADO**; el despachador correlaciona y asigna recursos.

Capturas operativas del sistema

EVIDENCIAS UX INTEGRADAS AL INFORME (FIGURAS 14–15B)



Stack Docker local · datos reales · [docs/informe-evidencias/](#)

Diseño UX/UI y Design System

SISTEMA VISUAL MONOCROMÁTICO INSTITUCIONAL

--REV-BG
#07111F

--REV-ORANGE
#F97316

--REV-SURFACE
#10233E

TIPOGRAFÍA
Inter · Segoe UI

COMPONENTE	USO EN REV
RevLogo	Identidad en shell y login
KpiCard / ModuleHub	Métricas y layout módulo
DegradedAlert	Modo información parcial
OperationalAmbient	Fondo cartográfico

Principios de diseño

- Paleta oscura → menos fatiga en sala de despacho
- Naranja único acento → jerarquía clara
- Glass cards + grid 8px → consola operacional
- CSS por módulo: `incidentes.css`, `zonas.css`, `portal.css`

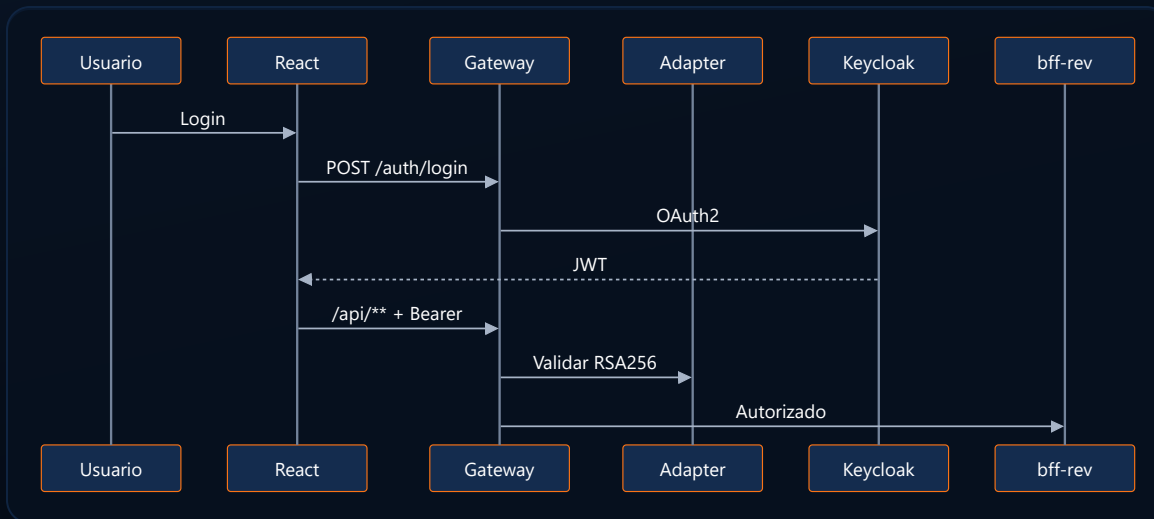
Roles y permisos

MATRIZ VERIFICADA — REALM KEYCLOAK **REV**



Seguridad

¿CÓMO PROTEGE REV LA INFORMACIÓN?

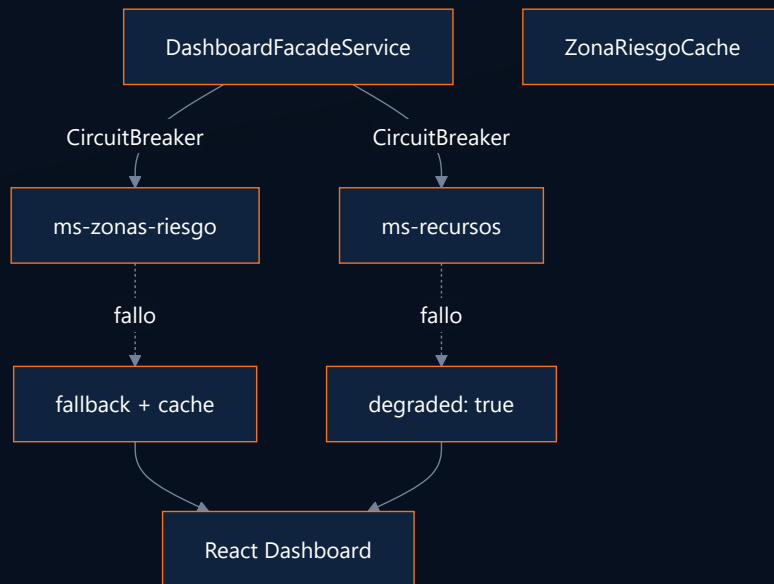


CAPA	MECANISMO
Identidad	Keycloak realm rev
Token	JWT RSA256
Perímetro	AuthenticationFilter
Público	/api/public/** acotado

Control en Gateway + adapter; MS sin **@PreAuthorize** (documentado).

Continuidad operacional y resiliencia

¿QUÉ OCURRE CUANDO UN SERVICIO FALLA?

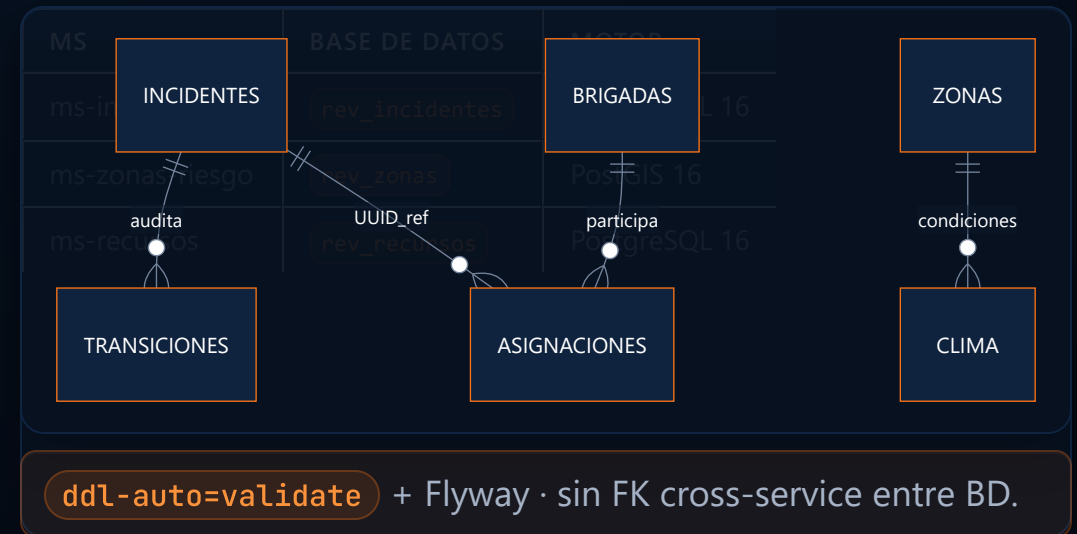


PARÁMETRO	VALOR
<code>slidingWindowSize</code>	10
<code>failureRateThreshold</code>	50%
<code>waitDurationInOpenState</code>	5s

UX: «Información parcial» — el despachador sigue operando con incidentes visibles.

Persistencia y base de datos

DATABASE PER SERVICE + FLYWAY (INFORME CAP. 11)

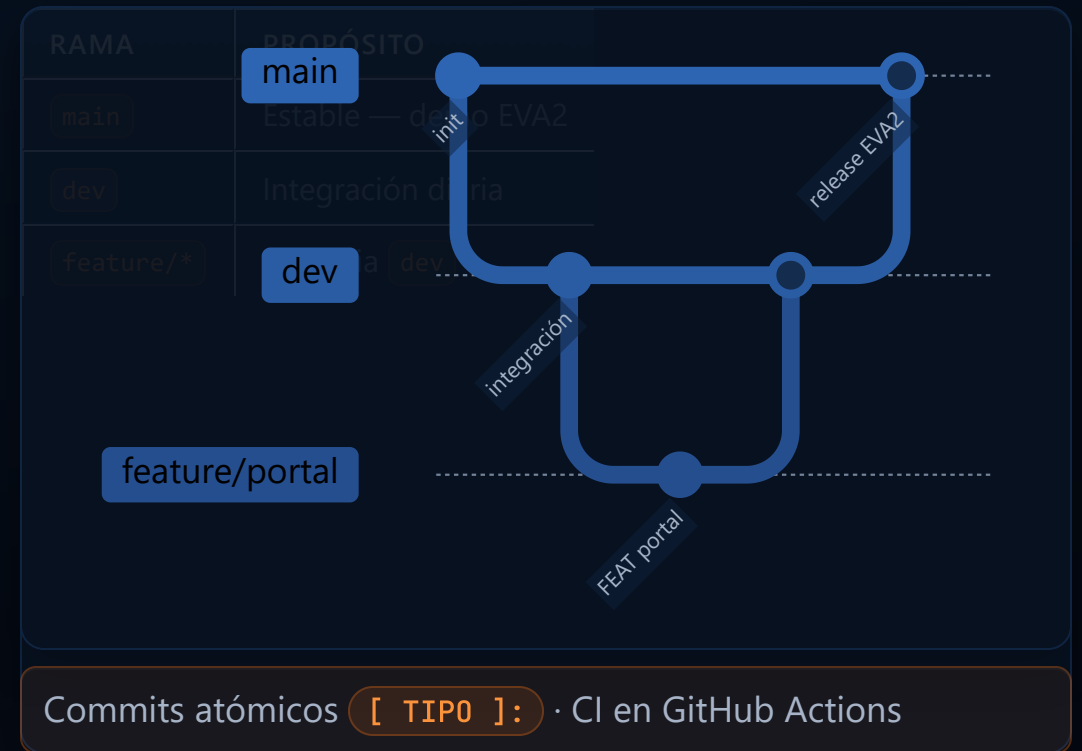


Observabilidad y trazabilidad

ESTADO ACTUAL VS ROADMAP (INFORME CAP. 10)



GITFLOW SIMPLIFICADO (INFORME CAP. 7)



Evidencias técnicas del informe

GALERÍA FIGURAS 1–20 — TRAZABILIDAD AL REPOSITORIO

FIG. 1–7

Patrones diseño

Cap. 5

FIG. 8–9

Git branching

Cap. 7

FIG. 10–12

Seguridad

Cap. 9

FIG. 13

Flyway SQL

Cap. 11

FIG. 14–16B

Capturas UX

Cap. 12–13

FIG. 17–20

Infra Docker

Cap. 13

Referencia: [informe-tecnico-integral-rev.html](#) — diagramas Mermaid, código y paginación PDF.

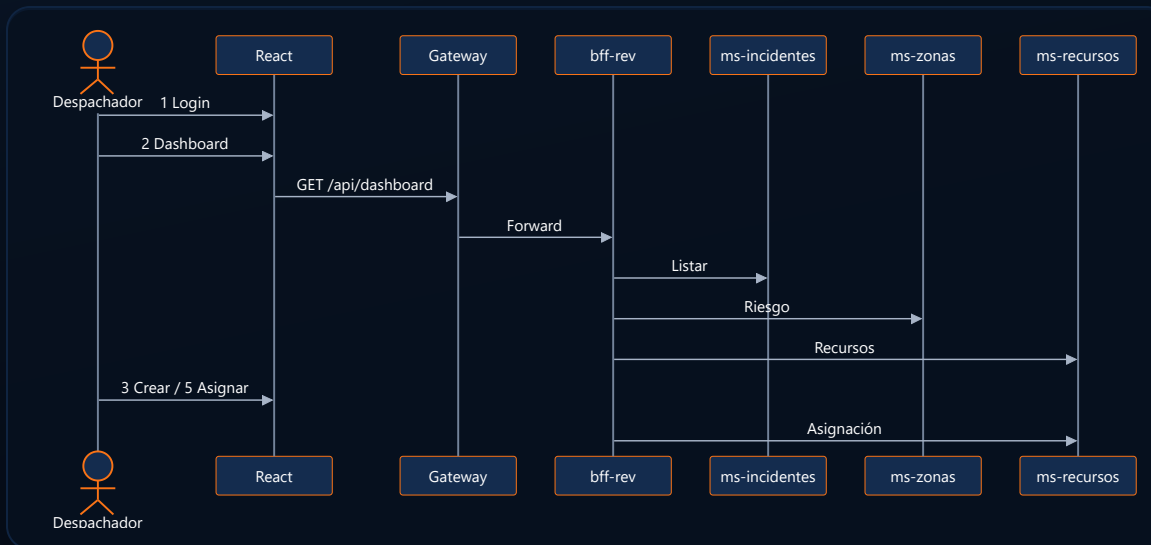
Tecnologías utilizadas

STACK VERIFICADO Y JUSTIFICACIÓN

CAPA	TECNOLOGÍA	DETALLE
Frontend	React + Vite + TS	React 18, Vite 5
UI	Bootstrap 5 + Icons	Grid accesible
Mapas	Leaflet	ZonasPage sin licencias
Backend	Java 21 + Spring Boot 4	Spring Cloud 2025.1
Resiliencia	Resilience4j 2.2.0	Circuit Breaker en BFF
BD	PostgreSQL 16 + PostGIS	3 bases aisladas
Infra	Docker Compose	12 servicios
Seguridad	Keycloak 24	realm rev
Monitor	Spring Boot Admin	:8099 vía Eureka

Flujo funcional del sistema

RECORRIDO OPERATIVO DE PUNTA A PUNTA



PASO	PANTALLA
1	LoginPage
2	DashboardPage
3	Modal incidentes
4	ZonasPage mapa
5	Asignar recurso

Ciudadano: PortalPage → POST público sin login.

Resultados obtenidos

TÉCNICOS · OPERACIONALES · MUNICIPALES

RESULTADOS TÉCNICOS

- Monorepo con 3 MS + BFF + Gateway + IAM
- 6+ patrones de diseño trazables a clases
- Circuit Breaker + cache aside operativos
- Arquetipo Maven custom documentado
- Tests en Factory, zonas y fallbacks BFF
- CI en GitHub Actions (`main`, `dev`)

RESULTADOS OPERACIONALES

- Dashboard unificado multi-fuente
- Portal ciudadano sin fricción
- Mapa de zonas de riesgo
- Asignación brigada/vehículo desde UI
- KPIs: activos, alto riesgo, con avisos
- Modo información parcial ante fallos

Beneficio municipal: coordinación más rápida, menor carga cognitiva del despachador y canal vecinal directo con mapa OSM.

Conclusiones

RESPUESTAS TÉCNICAS DE CIERRE

SOLUCIÓN MODERNA

Microservicios reales

Discovery · BFF · IAM · React

ARQUITECTURA ADECUADA

Picos · territorio · seguridad

PostGIS · JWT · Circuit Breaker

VALOR MUNICIPAL

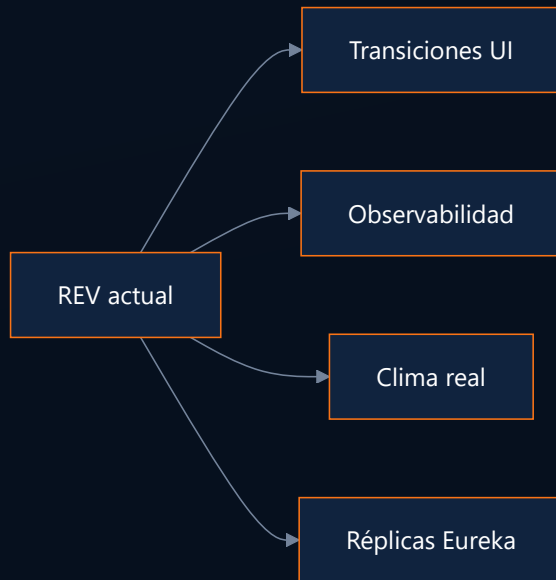
Conectividad que salva vidas

Despacho + terreno + comunidad

CRITERIO	DECISIÓN REV
Picos de crisis	MS escalables por dominio
Datos sensibles	Gateway perimetral + JWT
Fallos parciales	degraded + DegradedAlert

Evolución futura

PROYECCIONES DOCUMENTADAS (INFORME CAP. 14.4)



PRIORIDAD	EVOLUCIÓN
Alta	UI transiciones de estado
Alta	Mapa PostGIS avanzado
Media	CRUD zonas/recursos Admin
Baja	IoT climático real

Cada MS puede replicarse detrás de Eureka sin reescribir el frontend.

Conectividad que salva vidas

Resumen ejecutivo final

REV entrega una plataforma de emergencias municipal basada en microservicios Spring Cloud, React, Keycloak y Resilience4j.

ENTREGABLE EVA2	ARTEFACTO
Informe técnico integral	<code>informe-tecnico-integral-rev.html</code>
Frontend NPM	<code>frontend/rev-dashboard/</code>
BFF + 3 microservicios	<code>bff-rev</code> + 3 MS
Evidencias fig. 1–20	<code>docs/informe-evidencias/</code>
Git + CI	<code>main</code> / <code>dev</code> · GitHub Actions

¿Preguntas?

Red de Emergencia Valle · Duoc UC · DSY1106 · EVA2 · Mayo 2026